Khan *et al. Cybersecurity* (2026) 9:27
https://doi.org/10.1186/s42400-025-00457-3

Cybersecurity

RESEARCH Open Access

Privacy-preserving graph similarity search with attribute-based access control

Shawal Khan[1*], Shahzad Khan[2], Weisheng Si[1] and Bahman Javadi[1]

Abstract

Graph-structured data are integral to applications like social networks, biological systems, cybersecurity, and fraud detection. Outsourcing these data to public clouds offers scalability but raises privacy concerns, as encryption is required before outsourcing, making traditional graph similarity search and access control challenging. This paper presents a novel solution for privacy-preserving full graph similarity search with fine-grained access control in cloud environments. To the best of our knowledge, this is the first work to integrate privacy-preserving graph similarity search in a multi-user/multi-query setting with attribute-based access control (ABAC). This enables scalable and secure access in realistic, collaborative environments. The graph owner leverages the neural Graph2vec model to create feature indexes for encrypted graph data. Simultaneously, a secure transfer learning mechanism enables graph users to generate query feature indexes in the same latent space, ensuring privacy while accurately capturing the user's query intent. ABAC is employed to enforce flexible, fine-grained access policies. We conduct a formal security analysis under known-ciphertext and known-background threat models, demonstrating strong privacy guarantees. Experimental evaluations on real-world datasets show that our scheme achieves high semantic accuracy, lower search latency, and reduced storage overhead, outperforming existing approaches.

Keywords Graph similarity search, Secure transfer learning, Structure-aware search, Attribute-based access control

Introduction

Graphs are widely used to represent and analyze complex relationships and interactions among entities. In a graph, nodes represent entities, and edges represent the relationships between those entities. The graph data structure has applications in various research areas such as social networks, biological networks, and cybersecurity. In social networks, graphs are used to model user relationships and information flow, enabling tasks like community detection and influence maximization. In biological networks, they help represent complex interactions such as protein-protein interactions or gene regulatory networks. In cybersecurity, graph-based techniques are applied for intrusion detection, malware propagation modeling, and network anomaly detection. Given the complexity and size of real-world data in these domains, it is essential that any proposed scheme demonstrates its scalability and practicality. To this end, incorporating case studies or performance evaluations on real datasets, such as a Facebook friendship graph or a protein interaction dataset, would provide concrete evidence of the scheme's real-world applicability and robustness (Yan et al. 2004). Graph similarity search refers to comparing two graphs to find the similarities or differences in structure, node features, and connectivity patterns. Most often, individuals and organizations outsource their data to cloud servers to reduce the cost of storage, computing, and management (Sun et al. 2018). However, the data contain sensitive information, raising privacy concerns; hence, the data is encrypted before outsourcing to the

*Correspondence:
Shawal Khan
shawalsbbu@gmail.com; 22085153@student.westernsydney.edu.au
[1] School of Computer Data and Mathematical Sciences, Western Sydney University, Parramatta 2150, Australia
[2] Department of Information Security, National University of Science and Technology, Islamabad 44000, Pakistan

Springer Open

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

cloud server. The encryption process solves the problem of privacy and confidentiality, but at the same time, the traditional encryption process makes it impossible to query or retrieve the data from the database (Liu et al. 2014). Such a graph query, which requires the encoding of the entire graph structure, can have an application of its own but also provides building blocks for various complex tasks. In bioinformatics, for instance, embedding (vector representation) of the chemical compound could be used to detect their properties, such as anticancer and solubility (Yanardag and Vishwanathan 2015), and those of program call graphs could be deployed for malware detection (Huang et al. 2023). To enable privacy-preserving neighbor and adjacency search, Chase and Kamara (2010), based on the client/server model, introduced the notion of structure encryption for a graph database in a Single-GO/Single-GU (S/S) setting. The S/S setting means only a single user can encrypt and outsource the data to the cloud server as depicted in Fig. 1. To make the data accessible to more than one user, a secret key must be shared, and in the S/S setting, a secure channel is required to distribute the secret key used to access the outsourced document. This key sharing requires a secure channel and potentially leads to privacy issues and complex secret key management. Also, all of the contemporary searchable encryption (SE) schemes for graph-structured data operate on the S/S setting, which is the realization of symmetric searchable encryption (SSE) (Song et al. 2000). In the first work presented in Chase and Kamara (2010), the data owner encrypts the graph-structured data so that it can be efficiently queried for adjacency, neighbor, and sub-graph queries with the help of a specific token constructed with a shared secret key. When the user sends the query, the cloud server returns the encrypted search result without revealing the actual data and information about the query. Despite the above important types of graph queries, finding the graph similarity between two graph-structured data was not supported.

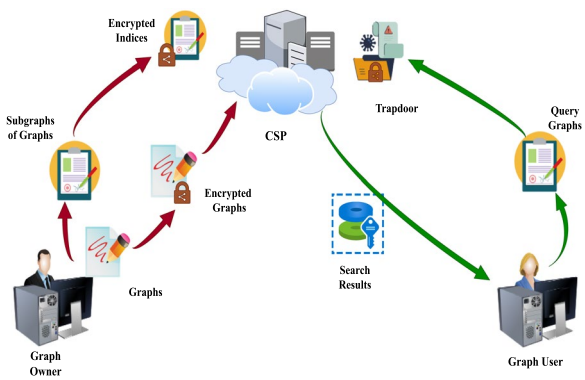


Fig. 1 Client/server based searchable encryption model

This secret key management in a Multi-GO/Multi-GU (M/M) setting is a challenging problem. In the M/M setting, multiple data owners can encrypt and outsource the data that can be accessible by multiple users. For instance, in shared cloud-based storage at a university, both the faculty members and students can encrypt and outsource the data with different access control policies. The data will be only accessible to authorized users whose attributes satisfy the access control policy. The M/M setting needs proper access control to make sure only authorized users can access the data. To enforce the access control policy, we leverage attribute-based access control. We also investigate using a graph-level embedding technique named Graph2vec (Narayanan et al. 2016) for vectorization of graph data. Inspired by the Doc2vec document-level embedding model, Graph2vec views the subgraphs in the graph as words and the entire graph as a document to learn the representation of a whole graph. The main expectation from this graph-level embedding is that a similar graph structure will reside close to each other in the vector space model. Specifically, given a graph-structured dataset, Graph2vec takes the set of all subgraphs around every node as its vocabulary. Subsequently, learn the distribution representation of individual graphs using the Graph2vec skip-gram training model. To this end, the graph-level neural embedding Graph2vec model is better suited for structure-aware search over encrypted graph-structure data. In this work, we leverage the Asymmetric scalar-product-preserving encryption (ASPE) (Wong et al. 2009) for encryption of graph feature and graph query vector. The ASPE preserves the special types of scalar products that are used in our proposed work for similarity calculation between feature and query vectors. That is, the GO trained the Graph2vec model over a specific graph-structure dataset and got a structure-aware index for each graph where the features in the index represent the subgraph of a data graph.

With the help of a secure transfer learning technique, GU utilizes the same pre-trained Graph2vec model to infer the structure-aware graph query vector as the trapdoor for search. The cloud server provider (CSP) performs the secure inner product operation between each data graph and graph query index during the search process. This operation measures the number of subgraph features in the graph query contained in the data graph, resulting in a similarity score. Afterwards, the CSP returns the top-k graph-structured data as the search result.

To enable convenient privacy-preserving graph similarity search in the M/M setting, this paper proposes a new Privacy-preserving Graph Similarity Search

(PGSS) scheme, which is combined with an Attribute-Based Access Control (ABAC) scheme. We name our entire scheme PGSS-ABAC. Inspired by the methodology introduced in our earlier work on document data structures (Khan et al. 2024), our PGSS part extends the framework in Khan et al. (2024) to address secure full graph similarity search. The secure transfer learning enables the data users to use the same pre-trained model for a better search experience.

To enforce robust access control, our ABAC part adopts Ciphertext-Policy Attribute-Based Encryption (CP-ABE) (Bethencourt et al. 2007). In CP-ABE the keys are associated with a set of attributes and documents and indexes are encrypted according to the access policy, a ciphertext can only be decrypted by a user if and only if the user's attribute list matches the embedded ciphertext's access policy. The attributes could be characteristics or properties that a user has, and they can represent roles, ages, and departments. The access policy is defined by the data owner during the encryption. The access policy is often represented by attributes, e.g., a policy might be (X AND Y) OR (B AND C), which means the user who wants to decrypt must either have both attributes X and Y or both B and C. It specifies the combination of attributes required for the decryption of the ciphertext.

To create a more expressive system, our proposed scheme can express a more complex access control policy with negative attributes, which can indicate the denial of certain attributes in the ciphertext (Ostrovsky et al. 2007). For instance, if the policy includes "Department:Computer Science" AND "Year=2024" in addition, we can also mention (NOT "Mathematics"), which means the ciphertext is not intended for the mathematics department. Besides, a detailed and comprehensive performance evaluation demonstrates its search precision and efficiency.

The contributions of this paper can be summarised as follows:

- We leverage the pre-trained Graph2Vec model and a transfer learning technique to transform the graph-structured data into high-dimensional latent space (Narayanan et al. 2017). After embeddings, we use ASPE encryption to calculate the cosine similarity between two encrypted vectors.
- We address the challenge of secret key distribution links with a K-Nearest-Neighbor-based SE Scheme with attribute-based access control, which ensures that only the authorized users designated by the dataset owner can access the data.
- Theoretical security analysis proves that privacy requirements under the known-ciphertext and known-background threat models are met in searching encrypted graph data.
- Detailed experimental study on four graph datasets demonstrates the efficiency of PGSS-ABAC in terms of semantic accuracy, search time, and storage cost.

In real-world applications, a multi-owner/multi-user searchable encryption scheme is essential for enabling secure and flexible data sharing over public cloud environments. In such schemes, multiple data owners can independently upload encrypted data, while authorized users can perform keyword searches without accessing the plaintext. This is especially useful in sectors like healthcare, where hospitals share encrypted patient records; collaborative research, where scientists securely share datasets; and enterprise systems, where departments manage confidential documents. These schemes ensure data privacy, fine-grained access control, and efficient search, addressing the needs of both secure collaboration and regulatory compliance in cloud-based systems.

The rest of the paper is organized as follows. Section 2 discusses related papers, covering their functionalities and limitations. Section 3 provides background information and mathematical primitives used in the paper. Section 4 presents the system model and the threat model. Section 5 details the algorithms in the proposed scheme and conducts complexity and security analysis. Section 6 reports the experimental results. Finally, Section 7 concludes this paper with future work.

Related work

The recent work named PrigSim presented in Wang et al. (2023) proposed a privacy-preserving graph similarity search. The authors used Graph Edit Distance as a similarity metric and a two-party additive secret sharing scheme. In terms of architecture, their scheme does not support multi-owner and also does not provide access control. Another related work presented in Wang et al. (2022) only supports exact and fuzzy keyword searching and lacks multi-user and access control. In contrast, our proposed work support multi-owner, access control and address the secure key distribution using attribute-based access control. Chase and Kamara (2010) was the first to introduce the idea of structured encryption. The secret key enables the construction of a specific token to query the encrypted structured data. Their work preserves the privacy of index, query, and graph features. The problem of subgraph search on the encrypted graph in the cloud was solved by Cao et al. (2011) by applying the filtering and verification mechanism. Yi et al. (2014) analyzed the privacy-assuring reachability query over encrypted data to preserve both the graph-structured and the query. He

et al. (2010) proposed privacy-preserving vertex reachability to preserve the edge privacy of the encrypted graph. Kundu et al. (2012) study various mechanisms to verify the authenticity of graph data without any information leakage. Gao et al. (2011) proposed computing the shortest distance with neighbourhood privacy assurance in the cloud computing paradigm. However, it exposed some distance and connection information between vertices. Mouratidis and Yiu (2012) proposed the first private information retrieval (PIR)-based framework for privacy-preserving shortest path computation. However, the PIR (Chor 1998) protocol causes high computation for query processing. The work of Karwa et al. (2011) presents various efficient algorithms to answer subgraph counting queries while guaranteeing vigorous privacy requirements. Fan et al. (2013) was the first to propose an authenticated subgraph query service using Merkle hash trees in the setting of data outsourcing. In his proceeding work, Fan et al. (2015) propose an optimized scheme for structure-preserving subgraph queries by relaxing the privacy requirements of the data graphs. The algorithm of Ullmann (1976) finds undirected subgraph isomorphism. Several algorithms (e.g, Quick SI Shang et al. 2008, VF2 Cordella et al. 2004 and Turbo$_{iso}$ (Han et al. 2013)) have been proposed to enhanced the Ullmann (1976) algorithm. However, the query in all these algorithms requires training on graph data while carrying structural information, making privacy preservation complicated. Zhang et al. (2020) used the Paillier cryptosystem to realize the secure minimum value and integer comparison operation on an accurate-constrained shortest distance query technique over an encrypted graph. Ge et al. (2021) deployed two cloud servers to realize search pattern privacy for shortest distance query in the industrial Internet of things (IIoT) environment. Du et al. (2020), with the help of customized primitive, can answer more complicated graph queries, including PageRank and dynamic updates beyond the shortest distance. Zhu et al. (2016) utilize an encryption mechanism with a stemming algorithm to perform a privacy-assured synonym query on an encrypted graph. Zuo et al. (2020) presented a privacy-assuring subgraph query service with authentication based on the Boneh-Goh-Nissim cryptosystem. All of the above studies revealed at least two shortcomings.

We summarize the existing work in the tabular form presented in Table 1 and categorize it based on its functionality, such as user multiplicity, matching technique, verifiability, encryption, and privacy. Firstly, all of these schemes are in the S/S setting; only the secret key holder can create a graph query for searching, significantly limiting the search scope. Secondly, the majority of the schemes do not provide verifiability of the results returned by the cloud server. Also, the graph query of these schemes can only inquire about subgraph/supergraph similarity under the constraint of a given threshold. However, these subgraph/supergraph similarity queries cannot be applied to full graph structure matching queries. These gaps, namely the lack of support for multi-user query capability and limited query expressiveness, directly motivate the design of the PGSS-ABAC framework, which enables full graph similarity search in a multi-user setting with fine-grained access control and the inherent verifiability capability of ABE.

Notations and background knowledge

This section provides background information about cryptographic primitives to help readers understand the techniques used in this and upcoming sections. In

Table 1 Comparative summary

Schemes	$\mathbb{F}_1$	$\mathbb{F}_2$	$\mathbb{F}_3$	$\mathbb{F}_4$	$\mathbb{F}_5$
Chase and Kamara (2010)	×	Neighbor query	×	Induced permutation	×
Cao et al. (2011)	×	Subgraph query	✓	Secure k-nearest neighbor(kNN)	✓
Yi et al. (2014)	×	Reachability query	×	Elgamal encryption	✓
He et al. (2010)	×	reachability query	×	Random perturbation	✓
Kundu et al. (2012)	×	reachability query	×	Secure k-nearest neighbor(kNN)	✓
Gao et al. (2011)	×	shortest distance query	×	1-neighbourhood-d-Radius	✓
Mouratidis and Yiu (2012)	×	shortest path query	×	k-anonymity	✓
Karwa et al. (2011)	×	Subgraph query	×	k-automorphism	✓
Fan et al. (2013)	×	Subgraph query	✓	Merkle hash tree	×
Fan et al. (2015)	×	Subgraph query	✓	Cyclic group based encryption (CGBE)	✓
Zhang et al. (2020)	×	shortest distance query	✓	Paillier cryptosystem	✓
Zuo et al. (2020)	×	Subgraph query	✓	Boneh-goh-nissim cryptosystem	✓

$\mathbb{F}_1$: Multi-owner/Multi-user setting; $\mathbb{F}_2$: Matching mechanism; $\mathbb{F}_3$: Verifiable; $\mathbb{F}_4$: Encryption technique; $\mathbb{F}_5$: Privacy

Table 2 Notation descriptions

Notations	Descriptions
$\mathbb{G}_1, \mathbb{G}_T$	Bilinear groups of order p.
$\mathcal{K}_{local}, \mathcal{DK}$	GU's local key and delegated key, respectively.
$\mathbb{Z}_p, \mathbb{Z}_p^*$	Finite fields.
γ, S	GO set of attributes and GU chosen set of attributes.
$G, \tilde{G}$	The plain-text and encrypted graph-structure dataset.
G_v	The plain-text index vector.
I, m	The encrypted index and number of features.
Q	Graph-structure search query for the GU.
$Q_v, \tilde{Q}_v$	The query feature vector and trapdoor vector.
M_w	Extracted Graph2Vec hidden layer weights matrix.
$\tilde{M}_w$	Secure weights matrix based on M_w.
M_1, M_2	Invertible matrices for inner product operation.
$\tilde{M}_1, \tilde{M}_2$	Secure invertible matrices based on M_1 and M_2.
$RoTR(x)$	Circular right shift notation of an argument x.
$LoTR(x)$	Circular left shift notation of an argument x.
A_c, V	Access control, and configuration vector.

Table 2, we give notations and their descriptions used throughout the paper.

Monotonic access structure

An attribute set $U = \{u_1, u_2, \ldots, u_n\}$ can form a monotonic collection $\mathbb{A} \subseteq 2^U$ if for any B in $\mathbb{A}$ where $B \subseteq C$, we also have $C \in \mathbb{A}$. When $\mathbb{A}$ is a collection of non-empty sets on U excluding the empty set; it is called a monotonic access structure. The sets in $\mathbb{A}$ are considered authorized, while any set not in $\mathbb{A}$ is an unauthorized set.

Decisional Bilinear Diffie-Hellman (DBDH) assumption

Suppose that a, b, c, and Z are randomly selected from $\mathbb{Z}_p$. Let $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$ be a bilinear map with generator $g \in \mathbb{G}$. The DBDH assumption, as proposed by Boneh and Boyen (2004), states that it is computationally infeasible for a probabilistic polynomial time adversary to distinguish between two tuples: $< g, g^a, g^b, g^c, e(g,g)^{abc} >$ and $< g, g^a, g^b, g^c, e(g,g)^z >$, where z is chosen randomly, with non-negligible advantage.

Neural graph embedding model

Graph2vec, inspired by Doc2vec (Le and Mikolov 2014), considers an entire graph as a document and its rooted subgraphs as words to learn the distribution representation of a whole graph. Let a graph G consists of n nodes with a user-specified degree d as input, Graph2vec gives the set of $n(d+1)$ rooted subgraphs denoted by

$$c(G) = \{sg_1^{(0)}, sg_2^{(0)}, \ldots sg_n^{(0)}, sg_1^{(1)}, sg_2^{(1)}, \ldots sg_n^{(1)}, \ldots, sg_1^{(n)}, sg_2^{(n)}, \ldots sg_n^{(n)}\} \quad (1)$$

These subgraphs are labeled IDs according to the Weisfeiler Lehman (WL) relabelling strategy. Further, given a sequence of subgraphs $\{c(G_1), c(G_2), \ldots c(G_n)\}$ for a set of graphs $\{G_1, G_2, \ldots G_n\}$, Graph2vec learns their m-dimensional vectorization of $f(G_i)$ for G_i using skip-gram model (Le and Mikolov 2014). The model tries to maximize the following probability by taking the jth subgraph sg_j in $c(G_i)$ in G_i context:

$$\sum_{j=1}^{n_i(d+1)} logPr(sg_j \mid G_i) \quad (2)$$

where, probability $Pr(sg_j \mid G_i)$ is defined as

$$\frac{exp(f(G_i).f(sg_j)}{\sum_{sg \in v} exp(f(G_i).f(sg))} \quad (3)$$

where v is the vocabulary set of subgraphs across all the graphs. After skip-gram training converges, semantically similar graphs are mapped to a similar position in the vector space.

We employ the Graph2Vec model for feature vector generation, utilizing its standard configuration, which assumes undirected and unweighted graph structures. Graph2Vec can be extended to accommodate directed and weighted graphs through appropriate modifications to its node labeling and substructure extraction mechanisms. In scenarios where the directionality or edge weights of the graph are essential for capturing structural information, we can adopt alternative graph neural network models, such as GraphSAGE, which inherently support directed and weighted graph representations and are better suited for preserving such nuanced structural attributes.

Similarity score

We use cosine similarity measurement to measure how similar the given graph structure is in their subject matter. It takes the vector orientation rather than its magnitude and avoids biases for different-length documents. Given two n-dimensional vectors, p, and q, the similarity score is given by:

$$Score(p,q) = \frac{p \cdot q}{|p||q|} = \frac{\sum_{i=1}^{|v|} p[i] \times q[i]}{\sqrt{\sum_{i=1}^{|v|} p^2[i]} \times \sqrt{\sum_{i=1}^{|v|} q^2[i]}} \quad (4)$$

Secure transfer learning

Transfer learning is widely used in machine learning tasks where a model trained on one task is reused for related but different tasks. This involves fine-tuning and modeling the architecture. For instance, searching for a similar subgraph over a graph-structured dataset to achieve this training of a model from scratch is not a trivial task and is inefficient, particularly for a small-size dataset. Therefore, we leverage transfer learning in parallel with a pre-trained Graph2Vec model trained on the large dataset to generate embeddings of graphs. The higher layers of pre-trained models are employed, while the lower layers of the model can be fine-tuned to fit the new task (Çakmak 2023). More specifically, the single hidden layer of Graph2vec is trained for a specific task without the need to get its final results. Instead, we aim to train the weights of the hidden layer, which are the "subgraph vectors" representation of the graph-structure data. Besides, we apply transformation techniques to the learned weights matrix with the help of configuration vector V to make it secure transfer learning.

Problem formulation and system model

Here in the section, we explain problem formulation, the system model, including the entities involved, and the threat model presented in Subsection 4.2 to prove the security of the proposed work. We consider a cloud server that stores an encrypted graph dataset containing sensitive information. We do not fully trust the cloud server, which may infer useful information that can lead to the full disclosure of the dataset. This work aims to solve the problem of querying encrypted outsourced data and ensure that only authorized users can query the dataset.

System model

Our proposed system model includes a cloud service provider (CSP), a trusted attribute authority (TAA), a graph owner (GO), and a graph user (GU). Figure 2 demonstrates the interaction among these entities, with their roles detailed as follows:

1. TAA constructs various keys, so its role is fully trusted in the system model. Specifically, the process creates public parameters *PK*, *MSK*, and *SK* for every authorized user based on a set of verified attributes.
2. The graph owner (GO) holds a set of graph-structured datasets $G = \{G_1, G_2, \ldots, G_n\}$ and seeks to outsource their encrypted versions $\tilde{G} = \{\tilde{G_1}, \tilde{G_2}, \ldots, \tilde{G_n}\}$ to a CSP while enabling authorized users (GU) to perform searches. The GO first trains a Graph2vec model on G to obtain m-dimensional feature vectors G_{v_i} for each graph G_i. Using the trained Graph-2vec model, the GO generates a single-layer weights matrix M_w for secure transfer learning. The feature vectors G_{v_i} are then encrypted into secure indices I_i,

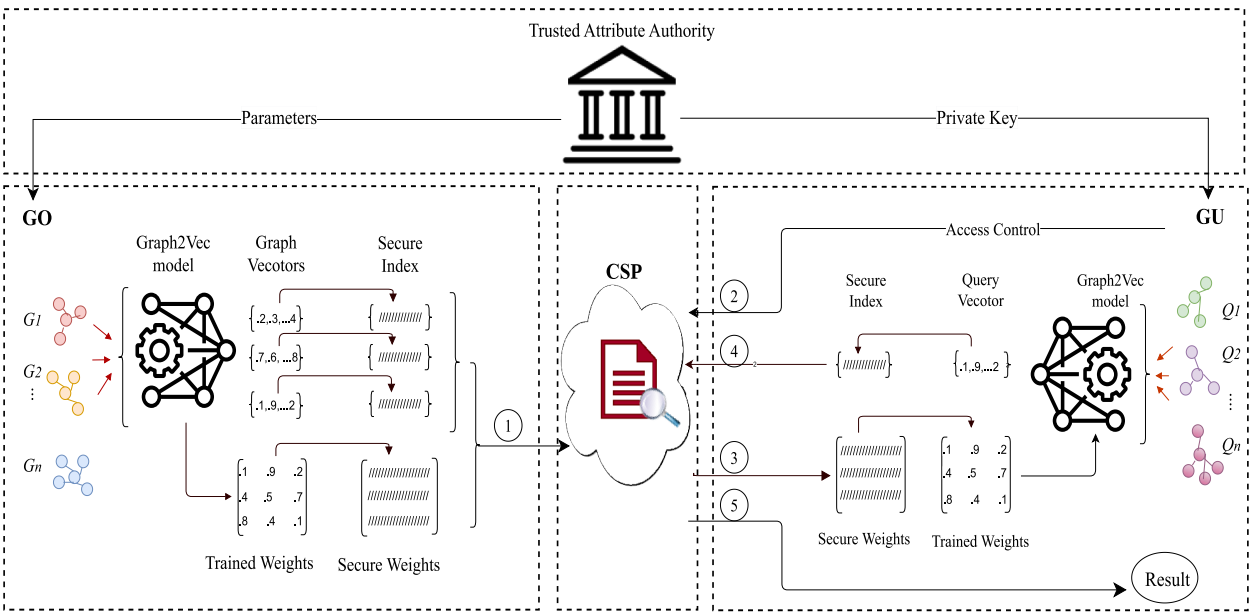


Fig. 2 Proposed system model: ① The GO employs Graph2vec to extract semantic feature vectors from graph data, encrypts them, and uploads both the encrypted features and the frozen model weights to the CS. ② The GU initiates an access-controlled authorization with the CS. ③ Upon successful authorization, the CS provides the frozen weights to the GU. ④ The GU uses the weights to generate an encrypted feature vector for its query graph and uploads it to the CS. ⑤ The CS performs a privacy-preserving similarity search and returns the matched results to the GU

which act as searchable indices. Finally, the GO uses attribute-based encryption (ABE) to enforce access control policies on $\tilde{G}$ and I for authorized GUs.

3. The CSP stores the GO encrypted datasets $\tilde{G}$ and processes secure queries on the encrypted index I for the GU. Communication with the GU occurs in two rounds: first, the CSP verifies the GU's authorization using an access control policy, returning $\perp$ if unauthorized. If authorized, the CSP then searches the secure indices I and delivers the ranked results to the GU.
4. Before initiating the search, the GU must undergo authorization from the CSP using an ABE access control policy. Once granted access, the GU receives intermediate ciphertext components ($\tilde{CT}$). Employing the Graph2vec model and its corresponding weights matrix (M_w), the GU computes the query feature vector (Q_v) based on the provided graph query (Q). Finally, the GU submits the trapdoor vector ($\tilde{Q}_v$) to retrieve ranked search results.

Threat model

Based on Cao et al. (2013) threat model, we adopted the following threat model according to the prior information to CS. In our proposed scheme, TAA is the only fully trusted entity. The CSP is assumed to be "honest-but-curious", which is the most common assumption in searchable encryption (SE) (Sun et al. 2013). Specifically, it is "honest" in the designated protocol operations. However, it is "curious" to analyze and infer additional information during interaction with the protocol. These assumptions are consistent with existing work; hence, we do not introduce any new threat model.

Known ciphertext threat model:

In this context, the CSP only has access to the outsourced encrypted data provided by users. The CSP could potentially conduct a ciphertext-only attack to recover the secure key and gain access to the plaintext of the encrypted graph datasets $\tilde{G}$, the search index vector I and the search trapdoor $\tilde{Q}_v$.

Known background threat model:

In this enhanced security model, the CSP is expected to possess additional contextual details, including the size of the graph-structured data, search results, and search patterns. The CSP can infer which graph-structured contains the specific subgraph structure with this additional information. That is to say, CSP can obtain the subgraph frequency distribution during the search outcome or can apply statistical attack (Delfs and Knebl 2015) to even recognize certain subgraphs.

Definition 1 The attribute-based encryption system ensures secure access control provided that any data user, even if acting maliciously and operating within polynomial time, gains no more than a negligible benefit in the context of the selective-set security challenge.

Required privacy-preserving guarantees

The proposed scheme should protect a curious CSP from learning sensitive information about encrypted graph-structured datasets, indexes, and queries. Specifically, the search scheme should provide the following privacy-preserving guarantees:

- Graph-structured data Confidentiality: The plaintext of graph-structured datasets G should be secure.
- Index privacy and query privacy: The index vector of the graph-structure G and search trapdoor vector $\tilde{Q}_v$ should be secured.
- Trapdoor unlinkability: The same search subgraph structure should not map to the identical trapdoor.
- Keyword privacy: The feature vector should not reveal the corresponding subgraphs in graph-structured, index, or trapdoor by analyzing the structure-aware correlation information.

Algorithms, complexity analysis, and security analysis

In this section, we introduce the algorithms in the proposed PGSS-ABAC scheme and conduct complexity and security analyses.

Algorithms in proposed PGSS-ABAC scheme

This subsection gives the pseudocodes of the algorithms and also explains them. The setup algorithm distributes the keys used for encryption and decryption operations. Once each entity receives the required keys, the EncGraph algorithm encrypts the data before outsourcing. A secret sharing scheme is employed at the KeyGen algorithm to enforce control over encrypted data. The trapdoor generated at the GU side with algorithm TDGen is then sent to the CSP, where it is used in the searching process. Finally, the search process involves access control verification and secure inner product operation.

1. **Setup**(λ) : Algorithm 1, carried out by the TAA, initiates the setup phase for our framework. The generator g is defined for a bilinear group $\mathcal{G}$ with a prime order p. The authority identifies a universal set of attributes $U = \{x_1, x_2, \ldots, x_n\}$ and computes $g_1 = g^{\alpha}$, $g_2 = g^{\beta}$, and $e(g,g)^{\alpha\beta}$, where $\alpha, \beta \in \mathbb{Z}_p^*$. Two polynomials, $p(x)$ and $q(x)$, each of degree n, are randomly selected such that $q(0) = \beta$. The publicly computable functions $U(x)$ and

$V(x)$ are defined by the authority as $g_2^{x^n} \cdot g^{p(x)}$ and $g^{q(x)}$, respectively. The use of Lagrange coefficients allows the computation of $g^{p(x)}$ and $g^{q(x)}$ using public key elements (Sahai and Waters 2005). Finally, the authority establishes the public key components PK and the master secret key MSK.

Algorithm 1 Setup

Input: λ.
Output: System's master secret key MSK and public parameters PK.
1. Form the universal set of attributes U= $\{x_1, x_2, \ldots, x_n\}$.
2. Create a multiplicative cyclic group $\mathbb{G}$ with prime order p.

3. Select elements α and β randomly from $\mathbb{Z}_p^*$, and compute $g_1 = g^\alpha$, $g_2 = g^\beta$, and $e(g,g)^{\alpha\beta}$.
4. The definition involves two polynomials, $h(x)$ and $q(x)$, both of degree n, where $q(0) = \beta$.
5. Two publicly computable functions T and V are selected, where $T : \mathbb{Z}_p \to \mathbb{G}$ maps x to $g^{x^n} \cdot g^{h(x)}$, and $V : \mathbb{Z}_p \to \mathbb{G}$ maps x to $g^{q(x)}$.
6. Calculate and publish PK as $= (g, g_1, g_2, \{g^{q(i)} \mid i \in n\}, \{g^{h(i)} \mid i \in n\}, e(g,g)^{\alpha\beta})$.
7. Accordingly, define the $MSK = (\alpha, \beta)$

2. **EncGraph**$(\mathcal{K}, PK, G)$: GO selects a symmetric key $\mathcal{K} \in \mathbb{Z}_p$ for encrypting the graph-structured dataset G and a randomly generated m-bit configuration vector V. $\mathcal{K}$ ensures the encryption of G, while V is used to scramble the Graph2vec hidden layer weights matrix M_w and secure inner product matrices. Additionally, GO chooses a random value $s \in \mathbb{Z}_p$.

GO trains the Graph2vec model on its own graph-structured dataset G to obtain m-dimensional feature vectors G_{v_i} for each G_i in G. These vectors serve as structure-aware plaintext indices, avoiding the need for extensive subgraph vectors in the dataset vocabulary. After training, GO freezes the model and extracts the hidden layer weights matrix M_w.

Next, GO transforms the plaintext indices G_{v_i} in G_v into secure counterparts I_i in I by employing a secure inner product operation (Algorithm 2, steps 6-13). G_{v_i} in G_v is also encrypted using $\mathcal{K}$ to produce $\{G'_v, G''_v\}$. The rows of M_w represent subgraph vector representations for each vocabulary subgraph in the graph-structured datasets. Normalized against each feature, these rows reveal structural relationships within specific subgraph data.

To obscure these relationships, GO utilizes configuration vector V to obfuscate them (Algorithm 2, steps 17-23), including operations involving transformation matrices M_1 and M_2. Additionally, GO computes access control components A_c using sets of negative and non-negative attributes $\gamma \in \mathbb{Z}_p^*$ to manage access control for the intended GU. GO maps configuration vector V to integer I_v, encrypting it along with symmetric key $\mathcal{K}$ for eligible GU. Finally, all ciphertext components are combined into CT and uploaded to the CSP.

Algorithm 2 EncGraph

Input: Symmetric key $\mathcal{K}$, Public parameters PK, and graph-structured dataset G
Output: Ciphertext Components CT
GO randomly generates $m \times m$ dimensional invertible matrices M_1 and M_2, m-bit configuration vector V, secret s and symmetric key $\mathcal{K}$ for graph-structured G
GO trained the Graph2Vec model with graph-structured G, get m-dimensional feature vector G_{v_i} for each G_i in G
Extracts hidden layer weight matrix M_w after freezing the own-trained Graph2vec model
for *each* $G_i \in G$ **do**
  if $V[j] = 1$ **then**
    $G'_{v_i}[j] = Rand()$ and $G''_{v_i}[j] = G_{v_i}[j] - G'_{v_i}[j]$
  end
  if *V[j]=0* **then**
    $G'_{v_i} = G''_{v_i}[j] = G_{v_i}[j]$
  end
end
Compute secure index I_i
$I_i = \{G'_{v_i}.M_1^T, G''_{v_i}.M_2^T\}$ and $I = I \cup I_i$
Using symmetric key $\mathcal{K}$ encrypt G_i to its $\tilde{G}_i$ and $\tilde{G} = \tilde{G} \cup \tilde{G}_i$
for *each* $j \in V$ **do**
  if $V[j] = 1$ **then**
    $\tilde{M}_w = \tilde{M}_w \cup RoTR(M_w[Row_i])$
    $\tilde{M}_1 = \tilde{M}_1 \cup RoTR(M_1[Row_i])$
    $\tilde{M}_2 = \tilde{M}_2 \cup RoTR(M_2[Row_i])$
  end
end
Compute access control for $\gamma \subset \mathbb{Z}_p^*$ as $Ac = (\gamma, C^{(2)} = g^s, \{C_x^{(3)} = T(x)^s \mid_{x\in\gamma}\}\{C_x^{(4)} = V(x)^s \mid_{x\in\gamma}\})$
Transform the configuration vector V into an integer I_v and encrypt it together with the symmetric key $\mathcal{K}$ as $C = (\{C^{(0)} = \mathcal{K} \cdot e(g_1, g_2)^s\}, \{C^{(1)} = I_v \cdot e(g_1, g_2)^s\})$
The complete ciphertext components are represented as $CT = (Ac, \tilde{M}_1, \tilde{M}_2, \tilde{G}, I, \tilde{M}_w, C)$

3. **KeyGen**(PK, MSK): Operated by the TAA, this phase establishes access control for GU's secret key components derived from system attributes U. If GO's required attribute list matches GU's attributes, access to the encrypted graph-structured data is granted. It uses public parameters PK, master secret key MSK, a non-monotonic access structure $NM(\mathbb{A})$, and GU's attributes S. The TAA employs a linear secret sharing scheme Π to generate shares λ_i for the system parameter α. For each attribute i, random values $r_i \in \mathbb{Z}_p$ and $t \in \mathbb{Z}_p$ are selected to create secret keys K_i and K_i' for attributes in S corresponding to non-negative and negative attributes, respectively. It provides the delegation key $\mathcal{DK} = (\{K_i, K_i'\} \mid_{i \in S})$ and a local secret key $\mathcal{K}_{local} = (\frac{1}{t})$.

Algorithm 3 *KeyGen*

Input: PK, MSK, and S.
Output: GU delegation key $\mathcal{DK}$ and local key $\mathcal{K}_{local}$
1. For each attribute $i \in S$, random selects values $r_i, t \in \mathbb{Z}_p^*$
for *each attribute* $i \in$ *negated attribute* **do**
  $K_i' = (K_i^{(3)} = g_2^{t\lambda_i + r_i}, K_i'^{(3)} = g_2^{s^{-1}\lambda_i + r_i}, K_i^{(4)} = V(x_i)^{r_i}, K_i^{(5)} = g^{r_i})$
end
for *each attribute* $\in$ *non-negative attribute* **do**
  $K_i = (K_i^{(1)} = g_2^{t\lambda_i}.T(x_i)^{r_i}, K_i'^{(1)} = g_2^{s^{-1}\lambda_i}.T(x_i)^{r_i}, K_i^{(2)} = g^{r_i})$
end
2. Sets the keys $\mathcal{K}_{local} = (\frac{1}{t}), \mathcal{DK} = (\{K_i, K_i'\} \mid_{i \in S})$

4. **Search**$_{\mathbf{A}}(CT, \mathcal{DK})$: This phase is run by the CSP and implemented in Algorithm 4 in our system model. After receiving the $\mathcal{DK}$ from GU, this algorithm operates in the following two phases:

Access Verification: To confirm the access authorization, the CSP needs to check whether the submitted $\mathcal{DK}$ attributes satisfies the attached attributes to the access control components. If it fails to satisfy, it aborts the search process; otherwise, it proceeds to process as follows:

For each negated attribute, it computes
$F_i = \frac{e(K_i'^{(3)}, C^{(2)})}{e(K_i^{(5)}, \prod_{x \in S}(C_x)^{(4)\sigma_x}).e(K_i^{(4)}, C^{(2)})^{\sigma_{x_i}}}$
Similarly, for each non-negated attribute, it computes
$F_i = \frac{e(K_i'^{(1)}, C^{(2)})}{e(K_i^{(2)}, C_i^{(3)})}$
Finally, the authorization is verified if
$\prod_{i \in I} F_i^{w_i} = e(g,g)^{\alpha\beta}$, else $F_i = \perp$

Ciphertext Pre-computation: This stage follows a similar procedure as described earlier, with the distinction lying in components $K_i'^{(1)}$ and $K_i'^{(3)}$. The process unfolds as follows:

For each negated attribute, it computes
$IC_i = \frac{e(K_i^{(3)}, C^{(2)})}{e(K_i^{(5)}, \prod_{x \in S}(C_x)^{(4)\sigma_x}).e(K_i^{(4)}, C^{(2)})^{\sigma_{x_i}}}$
Similarly, for the non-negated attribute, it computes
$IC_i = \frac{e(K_i^{(1)}, C^{(2)})}{e(K_i^{(2)}, C_i^{(3)})}$
Further, it computes $\prod_{i \in I} IC_i^{w_i} = e(g_2, g)^{ts\alpha}$ and set it to $\{I_R\}$. Finally, the CSP returns the $\tilde{CT}$ to the GU.

Algorithm 4 *Search*$_A(\mathcal{DK}, CT)$

Input: $\mathcal{DK}, CT$
Output: Precomputed ciphertext $\tilde{CT}$
for *each* x' *(negated attribute) in* S **do**
  $F_i = \frac{e(K_i'^{(3)}, C^{(2)})}{e(K_i^{(5)}, \prod_{x \in S}(C_x)^{(4)\sigma_x}).e(K_i^{(4)}, C^{(2)})^{\sigma_{x_i}}}$
end
for *each* x *(non-negated attribute) in* S **do**
  $F_i = \frac{e(K_i'^{(1)}, C^{(2)})}{e(K_i^{(2)}, C_i^{(3)})}$
end
if $\prod_{i \in I} F_i^{w_i} = e(g,g)^{\alpha\beta}$ **then**
  for *each* x' *(negative attribute) in* S **do**
    $IC_i = \frac{e(K_i^{(3)}, C^{(2)})}{e(K_i^{(5)}, \prod_{x \in S}(C_x)^{(4)\sigma_x}).e(K_i^{(4)}, C^{(2)})^{\sigma_{x_i}}}$
  end
  for *each* x *(non-negative attribute) in* S **do**
    $IC_i = \frac{e(K_i^{(1)}, C^{(2)})}{e(K_i^{(2)}, C_i^{(3)})}$
  end
  Compute $\prod_{i \in I} IC_i^{w_i} = e(g_2, g)^{ts\alpha}$ and set it to $\{I_R\}$
  $\tilde{CT} = (\{I_R\}, \tilde{M}_1, \tilde{M}_2, \tilde{M}_w, C^{(1)})$
else
  $F_i = \perp$
end

5) **TDGen**$(\mathcal{K}_{local}, Q, \tilde{CT})$: Implemented using Algorithm 5 within our system framework and executed by the GU, the process begins with the formulation of query Q as a graph-structured search query for the trapdoor vector. GU computes I_v from $e(g_2, g)^{\alpha s}$, transforming it into V. Using V, GU retrieves M_1, M_2, and M_w through an inverse shift row transformation. The Graph2vec model is then initialized with M_w, and Q generates Q_v, a m-dimensional feature vector. Finally, employing a secure inner product encryption operation, GU obtains the trapdoor vector $\tilde{Q}_v$ before transmitting it to the CSP.

Algorithm 5 TDGen

Input: $k_{local}, Q, \tilde{CT}$
Output: $\tilde{Q}_v$
Compute $\{I_R\}^{\mathcal{K}_{local}} = e(g_2, g)^{\alpha s}$.
Compute I_v to recover the integer and convert it into configuration vector V
$V = \frac{C^{(1)}}{e(g_2,g)^{\alpha s}} = \frac{I_v \times e(g_1,g_2)^s}{e(g_2,g_1)^s} = I_v$
for *each* $j \in V$ **do**
  if $V[j] = 1$ **then**
    $M_w = M_w \cup RoTL(\tilde{M}_w[row_i])$ $M_1 = M_1 \cup RoTL(\tilde{M}_1[row_i])$ $M_2 = M_2 \cup RoRL(\tilde{M}_2[row_i])$
  end
end
GU loads the Graph2vec model with trained weights matrix M_w and input subgraph query Q and obtained the query vector Q_v with m-dimensional features.
for *each* $i \in V$ **do**
  if $V[i] = 0$ **then**
    $Q'_v[i] = Rand()$ and $Q''_v[i] = Q_v[i] - Q'_v[i]$.
  end
  if $V[i] = 0$ **then**
    $Q'_v[i] = Q''_v[i] = Q_v[i]$.
  end
end
Using M_1 and M_2, compute secure search trapdoor $\tilde{Q}_v$ as
$\tilde{Q}_v = \{\tilde{Q}'_v, \tilde{Q}''_v\} = \{\tilde{Q}_v.M_1^{-1}, Q''_v.M_2^{-1}\}$

6) **Search**$_{\mathbf{B}}(\tilde{Q}_v, I)$: The CSP computes the inner product $I_i \cdot \tilde{Q}_v$ for each graph structure G_i to derive a ranked score R_{score}, then returns the top-k results to the GU along with $C^{(0)}$.

7) **Decryption**$(C^{(0)}, \mathcal{K}_{local})$: After receiving the top-k ranked graph-structure dataset, the GU obtained the symmetric key $\mathcal{K}$ as

$$\mathcal{K} = \frac{C^{(0)}}{e(g_2,g)^{\alpha s}} = \frac{\mathcal{K}.e(g_1,g_2)^s}{e(g_2,g)^{\alpha s}} = \frac{\mathcal{K}.e(g_1,g_2)^s}{e(g_1,g_2)^s}$$

We also leverage CB-Tree-based indexing in our proposed PGSS-ABAC scheme to enhance its search efficiency and security, of the proposed PGSS-ABAC scheme. Detailed explanations follow in subsequent subsections.

Time complexity of search

For each algorithm in our proposed PGSS-ABAC scheme, it is necessary to sequentially compute the inner product between the outsourced index vector I_i and the trapdoor $\tilde{Q}_v$, which is linear and hence insufficient for large datasets. Hence, we adopt CB-Tree as a search index, which helps us achieve sublinear search time. The CB-Tree is a counting-based self-adjusting binary data structure, and the tree structure is dynamically adjusted according to the most frequently accessed nodes closer to the root node. According to BULUT (2016), we let the array $N = [1, 2, \ldots n]$ store the CB-Tree, and each node $N[i]$ in CB-Tree has the following structure.

$$N[i] =< graphID, I_v, f_v >$$

where *graphID* represents the corresponding graph-structure G_i's ID, I_v denotes G_i's secure index vector, and f_v denotes the filter vector of $N[i]$ which filters out and stores the largest dimension value among all the child nodes. Also, for each element $N[i]$ in an array N, the following hierarchy is defined:

- The root node is denoted by $N[i]$.
- If $N[i]$ has both a right and left child node, they are denoted by $N[2i+1]$ and $N[2i]$, respectively.
- If $N[i]$ is not the root, its parent node is denoted by $N[i/2]$.

when the graph-structure dataset G contains n graphs, the height of the complete binary tree is $\lceil log_2(n+1) \rceil$. Algorithm 6 presents the formal construction process of the CB-Tree, which is characterized by its short height in comparison to other binary trees.

Algorithm 6 Construction of CB-Tree

Input: Dataset G with graph structure and secure index vector I
Output: Constructed CB-Tree
for *each* $i \in G$ **do**
  Create a node $N[i]$ where $N[i].graphID = i$ and $N[i].I_v = I_i$
end
for *each* $i \in [n, \ldots, 2, 1]$ **do**
  if $2i + 1 \leq n$ **then**
    $N[i].f_v = maxN[i].I_v, N[2i].f_v, N[2i+1].f_v$
  end
  else
    if $2i = n$ **then**
      $N[i].f_v = maxN[i].I_v, N[2i].f_v$
    end
    else
      $N[i].f_v = N[i].I_v$
    end
  end
end

The CSP stores the CB-Tree as a search index for its corresponding outsourced graph-structure dataset G. Starting from the root node $N[1]$, the search process utilizes the Greedy Depth First Search algorithm to traverse the nodes recursively. We describe the similarity score calculation and search result list in Section 3.3 and are denoted by function score () and R, respectively. Using the filter vector f_v, we need to store R in descending order after each new candidate graph is added. The detailed working mechanism is presented in Dai et al. (2019) and denoted by Latent Dirichlet Allocation (LDA) LDA_{search} algorithm. They leverage the LDA topic model, which is used for semantic-aware multi-keyword search. The LDA is first proposed for the NLP task for topic modeling to discover the semantic features. For a brief description, let $\tilde{Q}_v$ be the submitted trapdoor and $R[k]$ is the lowest score in list R; the algorithm works as follows:

1. The first step is to compute the score $(N[i].f_v, \tilde{Q}_v)$, and if it is less than the score $(R[k], \tilde{Q})$, the scheme will not proceed searching in that subtree.
2. The next step is to compute the score $(N[i].I_v, \tilde{Q}_v)$, and if it is less than the score $(R[k], \tilde{Q})$, the corresponding G_i will be inserted into R.
3. If the node $N[i]$ has a child node, the scheme will traverse its child nodes $N[2i]$ and $N[2i+1]$. Eventually, the CS will return the encrypted graph-structured datasets $\tilde{G}$ in R to GU.

Thus, by adopting the CB-Tree, the time complexity of the proposed PGSS-ABAC scheme is reduced to $O(log(n))$ from $O(n)$ for n graphs.

Security improvement

In our proposed PGSS-ABAC scheme, the splitting procedure of configuration vector V guarantees the construction of different trapdoor $\tilde{Q}_v$ for the same graph-structures search query Q. However, the ranked search score stays unchanged for the same search query Q. This liability enables the CSP to relate these query requests by inferring the ranked search score. Thus, to obfuscate this likability due to exact ranked equality, we insert some tunable e_b (error bits) in index graph-structured vector G_v and trapdoor $\tilde{Q}_v$ generation. As a result, this could decrease the search accuracy. However, a tradeoff between privacy and precision is adjustable from e_b.

To implement the CB-Tree-based index, the algorithms Build CB-Tree and GDF-Search are called inside EncGrpah and $Search_B$ algorithms, respectively. Hence, the scheme algorithms remain the same except for the algorithms mentioned as follows:

EncGraph(K, PK, G): GO first extends the m- dimensional matrices M_1 and M_2 to $(m+e_b) \times (m+e_b)$ -dimensional. It also extends m-bit configuration vector V to $(m+e_b)$-bit vector. Then, each graph-structured vector G_{v_i} is set to $(m+e_b)$-dimensional. The resultant vector $G_{v_i}, j = 1, 2, \ldots e_b$ is set to a random number ξ_j. Next, it calls algorithm 6 to construct a secure index tree $\tilde{N}$ based on I. Finally, the encrypted graph-structured datasets $\tilde{G}$ along with secure index tree $\tilde{N}$ is outsourced to the CSP.

TDGen(k_{local}, $\tilde{Q}$, $\tilde{CT}$): Query vector Q_v is also extended to $(m+e_b)$-dimensional vector. However, the values are randomly and independently set to 1 or 0.

Search($\tilde{Q}_v, \tilde{N}$): It calls the GDF-Search algorithm on secure CB-Tree index $\tilde{N}$ to provide efficient search results. However, the final similarity score for $\tilde{N}_i$ is now obfuscated to $Q_{v_i}.Q + m_{\xi_\upsilon}$, where $\upsilon \in \{j | Q[m+j] = 1\}$. Therefore, the privacy of the search query is enhanced.

Correctness of access control

Algorithm $Search_A$ outlines the procedure for the CSP to validate the authorization by verifying the equation below:

$$\Pi_{i \in I} F^{w_i} i = e(g,g)^{\alpha\beta} \tag{5}$$

within this framework, we designate authorized entities as $\tilde{\gamma} = N(\gamma) \in \mathbb{A}$. Let I denote the set of indices i where $\breve{x}$ belongs to $\tilde{\gamma}$, and $\{w_i \in \mathbb{Z}_p \mid i \in I\}$ represents a series of constants. Where $\breve{x}$ is a generalized representation of an attribute. It can refer to either the positive attribute or the negative attribute. Each valid share λ_i of the secret α, using the Π protocol, adheres to the equation $\sum_{i \in I} w_i \lambda_i = \alpha$. For each attribute $x_i \in \gamma$ (where $\breve{x}_i \in \gamma'$), the CSP calculate this equation accordingly.

$$\begin{aligned} F_i &= \frac{e(K_i'^{(1)}, C^{(2)})}{e(K_i^{(2)}, C_i^{(3)})} \\ &= \frac{e(g_2^{s^{-1}\lambda_i}.T(x_i^{r_i}), g^s)}{e(g^{r_i}, T(x_i)^s)} \\ &= \frac{e(g_2,g)^{\lambda_i} \times e(T(x_i)^{r_i}, g^s)}{e(g^{r_i}, T(x_i)^s)} \\ &= \frac{e(g_2,g)^{\lambda_i} \times e(T(x_i), g)^{r_i s}}{e(T(x_i), g)^{r_i s}} \\ &= e(g_2,g)^{\lambda_i} \end{aligned}$$

For attributes x_i that are not in γ (hence $\breve{x}_i$ is in γ'), we define γ_i as $\gamma \cup \{x_i\}$. Next, we apply Lagrange interpolation to the elements in the set γ_i to find the coefficients $\{\sigma_x\}_{x \in \gamma_i}$, ensuring that $\sum_{x \in \gamma_i} \sigma_x q(x) = q(0) = \beta$. The computation by the CSP continues as outlined below:

$$
\begin{aligned}
F_i &= \frac{e(K_i'^{(3)}, C^{(2)})}{e(K_i^{(5)}, \prod_{x\in S}(C_x)^{(4)^{\sigma_x}}).e(K_i^{(4)}, C^{(2)})^{\sigma_{x_i}}} \\
&= \frac{e(g_2^{s^{-1}\lambda_i + r_i}, g^s}{e(g^{r_i}, \prod_{x\in\gamma}(V(x)^s)^{\sigma_x}).e(V(x_i)^{r_i}, g^s)^{\sigma_{x_i}}} \\
&= \frac{e(g_2^{s^{-1}\lambda_i}.g_2^{r_i}, g^s)}{e(g^{r_i}, \prod_{x\in\gamma}(V(x)^s)^{\sigma_x}).e(V(x_i)^{r_i}, g^s)^{\sigma_{x_i}}} \\
&= \frac{e(g_2,g)^{\lambda_i} \times e(g_2,g)^{r_i s}}{e(g^{r_i}, g^{s\sum_{x\in\gamma}\sigma_x q(x)}).e(g^{r_i\sigma_{x_i}q(x_i)}, g^s)} \\
&= \frac{e(g_2,g)^{\lambda_i} \times e(g,g)^{r_i s\beta}}{e(g,g)^{r_i s\sum_{x\in\gamma'}\sigma_x q(x)}} \\
&= \frac{e(g_2,g)^{\lambda_i} \times e(g,g)^{r_i s\beta}}{e(g,g)^{r_i s\beta}} \\
&= e(g_2,g)^{\lambda_i}
\end{aligned}
$$

Finally,

$$
\begin{aligned}
\Pi_{i\in I} F_i^{w_i} &= e(g_2,g)^{\alpha} \\
\Pi_{i\in I} F_i^{w_i} &= e(g,g)^{\alpha\beta}
\end{aligned}
$$

Security analysis

The security robustness of the proposed scheme relies on the threat models outlined in Subsection 4.2. The initial two threats are examined concerning the algorithm functionality in alignment with privacy criteria. To address the access control threat, the security is analyzed through a game-based framework involving an interaction between an adversary $\mathcal{A}$ and a challenger $\mathcal{C}$.

Semantic search security

Theorem 1 *The PGSS-ABAC provides privacy to the graph-structured datasets.*

Proof

In the proposed scheme of PGSS-ABAC, the graph-structured datasets are encrypted by symmetric encryption (such as AES) with the secure key $\mathcal{K}$. Further, the secure exchange of this symmetric key is done with the help of an attribute-based access control policy. The correctness proof of access control policy is provided in Subsection 5.4. Hence, the privacy of graph-structured data is maintained since $\mathcal{K}$ is protected from CSP. □

Theorem 2 *Privacy of subgraph features in CB-Tree-based search index I and trapdoor $\tilde{Q}_v$ is maintained from curious CSP.*

Proof

In both the known-ciphertext and known background threat models, the CSP has the encrypted matrices $(\tilde{M}_1, \tilde{M}_2)$. The objective of the curious CSP is to recover the graph-structured index vector G_v and transformation matrices M_1 and M_2. According to the procedure of our proposed scheme, the Algorithm 2 first uses the configuration vector V to split G_v into two vectors randomly $\{G'_v, G''_v\}$ and then uses the transformation matrices to generate secure search indexes I. According to Algorithm 2, we have two equations for each graph-structured G_i in G_v

$$
\begin{aligned}
G'_{v_1}.M_1^T &= \tilde{I}_1 \\
G''_{v_2}.M_2^T &= \tilde{I}_2
\end{aligned}
$$

In order to recover G_v and $(\tilde{M}_1, \tilde{M}_2)$, the CSP needs to solve the above equations. There are $2m^2$ unknown elements in $(\tilde{M}_1, \tilde{M}_2)$ and $2mn$ unknown elements in $\{G'_{v_1}, G''_{v_2}\}$. However, only $2mn$ known equations are available, which are less than the total unknown equations $2m \times (m+n)$. Hence, CSP is not able to solve the above equations (Wong et al. 2009). According to Algorithm 5, the trapdoor is generated in a similar procedure. That is to say, the subgraphs in the query feature vector are also securely obfuscated. □

Theorem 3 *Our proposed scheme preserved trapdoor unlinkability.*

Proof

The GU uses the Graph2vec model to get the graph query vector Q_v for graph-structured data. Further, it splits Q_v into two vectors $\{Q'_v, Q''_v\}$ using the configuration vector V. For a given graph query vector Q_v, its encryption procedure is as follows:

if $V[i] = 0$, then $Q'_v[i] = Rand()$ and $Q''_v[i] = Q_v[i] - Q'_v[i]$. if $V[i] = 0$, then $Q'_v[i] = Q''_v[i] = Q_v[i]$. Finally, these vectors are encrypted using transformation matrices M_1 and M_2 as $\{Q'_v.M_1^{-1}, Q''_v.M_2^{-1}\}$. Thus, with the secure random generator function Rand(), the GU would get a distinct trapdoor vector even for the same graph-structured data. However, the CSP is still able to associate the different search requests due to the ranked search result list. To distort this linkability, we introduced error bits in the PGSS-ABAC to distort the resultant search results. Hence, trapdoor unlinkability is achieved. □

Theorem 4 *The privacy of the underlying graph-structured data is preserved in search index I and trapdoor $\tilde{Q}_v$.*

Proof

In theorem 2, in the known ciphertext threat model, has proved the privacy of CB-Tree-based search index I and trapdoor $\tilde{Q}_v$. This subsequently preserved the privacy of the underlying graph-structured data in the known ciphertext threat model. Assuming in the known-background threat model, our scheme uses the graph2vec model to express the graph-structured as a feature vector. Similarly, the GU using the same trained neural model expresses the graph query as a query feature vector. These graphs utilized as vector features do not preserve any structural meaning. The dot product of these feature vectors serves as the relevance score. Thus, the direct relationship between the queried graph and the relevance score is obfuscated. Therefore, the graph-structured privacy is preserved. □

Access control security

Theorem 5 *Assuming the hardness of the decisional Bilinear Diffie-Hellman (BDH) problem, no probabilistic polynomial-time (PPT) adversary is capable of compromising the access control mechanisms embedded in the EncGraph algorithm, even when presented with a challenge access structure γ.*

Proof

Suppose there exists an opponent $\mathcal{A}$ capable of extracting access control elements from the EncGraph Algorithm with a significant success probability ϵ. In that case, we can design a simulator $\mathcal{B}$ that achieves an advantage of $\frac{\epsilon}{2}$ in solving the decisional Bilinear Diffie-Hellman (BDH) problem. The challenger $\mathcal{C}$ begins by randomly choosing four elements a, b, c, z from $\mathbb{Z}^*p$. It then defines a bilinear pairing e between groups $\mathbb{G}$ and $\mathbb{G}T$. Before interacting with $\mathcal{B}$, $\mathcal{C}$ secretly generates a random bit $\mu \in 0, 1$. Depending on μ, $\mathcal{C}$ assigns $\mathbb{Z}\mu$ as follows: if $\mu = 0$, it computes $\mathbb{Z}\mu = e(g,g)^{abc}$; otherwise, it sets $\mathbb{Z}_\mu = e(g,g)^z$.

Init: The adversary $\mathcal{A}$ is initiated by the simulator $\mathcal{B}$. During this phase, $\mathcal{A}$ chooses a target attribute set γ consisting of d elements.

Setup: The simulator $\mathcal{B}$ calculates the public parameters by setting $g^\alpha = A$ and $g^\beta = B$, where α and β are implicitly defined as a and b respectively. A random polynomial $f(x)$ of degree d is selected by $\mathcal{A}$, which also defines another polynomial $U(x)$ of the same degree. For attributes within the set γ, $U(x)$ takes the value $-x^d$, while for all other attributes it is assigned arbitrary values different from $-x^d$. Next, $\mathcal{B}$ establishes two additional polynomials: $h(x) = \beta U(x) + f(x)$ and $q(x)$. For each attribute x_i in $\gamma = x_1, ..., x_d$, random elements θ_{x_i} from $\mathbb{Z}p$ are chosen, with $q(x_i)$ set to θx_i and the condition that $q(0) = \beta$ maintained. The public key components transmitted to $\mathcal{A}$ include the terms $g^{q_i} i \in [1, d]$ and $g^{h_i} = g_2^{u_i} g^{f_i} i \in [1, d]$. For simplicity in representation, the expression $T(x) = g_2^{x^d + U(x)} g^{f(x)}$ is used in the construction.

Phase 1: $\mathcal{A}$ makes multiple queries for various access control structures, intentionally omitting γ. Each request involves secret key components where $\tilde{\mathbb{A}}(\gamma) = 0$, with $\tilde{\mathbb{A}}$ representing the negation $NM(\mathbb{A})$ of some monotonic access structure $\mathbb{A}$ over attribute set S using linear secret sharing scheme Π. For attribute negation mapping in this simulation, we proceed as follows:

First, we establish $\alpha = a$ as the secret. We then randomly select a vector $\mathbf{v} = (\nu_1, \nu_2, \ldots, \nu_{n+1}) \in \mathbb{Z}_p^{n+1}$. Since $(1, 0, \ldots, 0)$ is linearly independent from $M_{\gamma'}$, we can compute (Anton and Rorres 2013) a vector $\mathcal{W} = (\mathcal{W}_1, \ldots, \mathcal{W}_{n+1})$ where:

- $(1, 0, \ldots, 0) \cdot \mathcal{W} = \mathcal{W}_1 = 1$
- $M_{\gamma'} \cdot \mathcal{W} = \overrightarrow{0}$

Here $M_{\gamma'}$ denotes the submatrix of M corresponding to attributes in γ'. We define the uniformly distributed vector $\upsilon = \nu + (a - \nu_1)\mathcal{W}$ with the constraint $\upsilon_1 = a$. The shares $\lambda = M_\upsilon$ are computed such that $\lambda_i = M_i \upsilon = M_i \nu$ for all $x_i \in \gamma'$, making them independent of a.

For negated attributes $\breve{x}_i = x'_i$:

- When λ_i depends on a (iff $x_i \in \gamma$):
  - Select random r'_i and $t \in \mathbb{Z}_p$
  - Set $q(x_i) = \theta_{x_i}$ and $r_i = -t\lambda_i + r'_i$
  - Output key components:

$$K_i^{(3)} = g_2^{r'_i}$$
$$K_i^{(4)} = g^{\theta_{x_i}(-t\lambda_i + r'_i)}$$
$$K_i^{(5)} = g^{-t_{\lambda_i} + r'_i}$$

- When λ_i is independent of a (iff $x_i \in \gamma$):

– Choose random $r_i \in \mathbb{Z}_p$
– Output:

$$K_i^{(3)} = g_2^{t\lambda_i + r_i}$$
$$K_i^{(4)} = V(x_i)^{r_i}$$
$$K_i^{(5)} = g^{r_i}$$

For non-negated attributes $\breve{x}_i = x_i$:

- When λ_i is independent of a (iff $x_i \in \gamma$):

– Select random $r_i \in \mathbb{Z}_p$
– Output:

$$K_i^{(1)} = g^{t\lambda_i} \cdot T(x_i)^{r_i}$$
$$K_i^{(2)} = g^{r_i}$$

- When λ_i depends on a (iff $x_i \notin \gamma$):

– Let $g_3 = g^{\lambda_i}$
– Choose random $r_i' \in \mathbb{Z}_p$
– Output:

$$K_i^{(1)} = g_3^{\frac{-f(x)}{x_i^d + u(x_i)}} \left(g_2^{x_i^d + u(x_i)} \cdot g^{f(x)} \right)^{r_i'}$$
$$K_i^{(2)} = g_3^{\frac{-1}{x_i^d + u(x_i)}} g^{r_i'}$$

Challenge: The adversary $\mathcal{A}$ chooses two messages M_1 and M_2 of identical length and requests their encryption from $\mathcal{B}$. The simulator $\mathcal{B}$ then performs the following steps:

1. Randomly selects a bit $\nu \in \{0,1\}$ 2. Generates the ciphertext as:

$$AC = \begin{pmatrix} A^{(1)} = M_\nu \times Z, \\ A^{(2)} = C, \\ \gamma, \{A_x^{(3)} = C^{f(x)} \mid x \in \gamma\}, \\ \{A_x^{(4)} = C^x \mid x \in \gamma\} \end{pmatrix}$$

Depending on the challenger $\mathcal{C}$'s earlier coin flip μ, two distinct scenarios occur:

- **Case** $\mu = 0$: Here $Z = e(g,g)^{abc}$. From $\mathcal{A}$'s perspective, the ciphertext AC appears identical to a legitimate encryption of message M_ν under the original scheme.
- **Case** $\mu = 1$: In this scenario, $Z = e(g,g)^z$ where z is randomly chosen. The component $A^{(1)} = M_\nu \times e(g,g)^z$ becomes uniformly random in $\mathbb{G}_T$, making the ciphertext AC information-theoretically independent of the message M_ν from $\mathcal{A}$'s viewpoint.

Phase 2: $\mathcal{B}$ maintains consistency by executing the simulation according to the previously defined Phase 1 protocol.

Guess: The adversary $\mathcal{A}$ produces its guess ν' for the value ν.

When $\nu = \nu$, two cases emerge:

- If $\mu' = 0$, this implies a random 4-tuple was provided
- If $\mu' = 1$, this indicates a BDH-tuple was given

The probability distribution is analyzed as follows:

Case $\mu = 1$**:** $\mathcal{A}$ obtains no information about ν, resulting in:

$$Pr[\mu = 1 \mid \nu' \neq \nu] = \frac{1}{2} \tag{6}$$

The simulator's strategy of choosing $\mu' = 1$ when $\nu \neq \nu'$ yields:

$$Pr[\mu' = \mu \mid \mu = 1] = \frac{1}{2} \tag{7}$$

Case $\mu = 0$**:** $\mathcal{A}$ observes a proper encryption of M_ν. Given $\mathcal{A}$'s assumed advantage ϵ, we have:

$$Pr[\mu = 0 \mid \nu' = \nu] = \frac{1}{2} + \epsilon \tag{8}$$

The simulator's decision to set $_{\mu' = 0}$ when $_{\nu = \nu'}$ produces:

$$Pr[\mu' = \mu \mid \mu = 0] = \frac{1}{2} + \epsilon \tag{9}$$

Combining equations (7) and (9), $\mathcal{B}$'s total advantage in the decisional BDH game becomes:

$$\begin{aligned} &| \tfrac{1}{2} Pr[\mu' = \mu | \mu = 0] + \tfrac{1}{2} + Pr[\mu' = \mu | \mu = 1] - \tfrac{1}{2} | \\ &= | [\tfrac{1}{2}(\tfrac{1}{2} + \varepsilon) + \tfrac{1}{2} \cdot \tfrac{1}{2}] - \tfrac{1}{2} | \\ &= \tfrac{\varepsilon}{2} \end{aligned} \tag{10}$$

□

Experimental study

In this section, we report a detailed experimental evaluation. All the experiments were conducted on Windows 10 with an Intel Core (TM) i7-3770 CPU. For platform realization, we used Python in the Gensim framework Řehůřek and Sojka (2010). To the best of our knowledge, since none of the existing schemes simultaneously

Table 3 Dataset statistics

Datasets	No of graphs	Avg Nodes
PTC	344	25.5
MUTAG	188	17.7
NCI1	4110	29.8
PROTEINS	1113	39.1

Table 4 Default parameters

Parameters	m	*n*	h	k
Values	1024	188	10	10

consider deep graph-level embedding and privacy-preserving search in the M/M setting, we present a comparative analysis for different settings under three aspects. Especially we compare the search time and storage cost with Cao et al. (2011), Zhu et al. (2019), and Liang and Zhao (2017) since they constitute full graph feature vectors, and compare semantic precision with scheme node2vec (Grover and Leskovec 2016) and Sub2Vec (Adhikari et al. 2017) since their full graph representation is straightforward. Four benchmark graph datasets, namely MUTAG[1], PTC[2], PROTEINS[3], and NCI1[4], are used in the experiment. These datasets belong to bio and chemoinformatics, and the specification is given in Table 3. In the experiments, m, h, n, and k denote the number of features, nodes in the graph search query, total graphs, and required graphs. Its default values are shown in Table 4.

We select the default experimental parameters based on practical considerations relevant to both performance and real-world applicability. Specifically, the feature dimension $m = 1024$ is chosen to provide a rich semantic representation of the graph structures while maintaining computational efficiency, as it is a commonly used size in neural embedding models like Graph2vec. The query graph size $n = 188$ reflects the typical number of nodes observed in the dataset, ensuring realistic and representative performance evaluation. The batch size $h = 10$ captures a moderate number of concurrent queries, simulating multi-user scenarios and helping assess the scheme's behavior under practical workloads. Finally, the retrieval size $k = 10$ aligns with standard practice in similarity search tasks, providing a top-k result set that balances informativeness and user interoperability.

[1] https://huggingface.co/datasets/graphs-datasets/MUTAG

[2] https://github.com/wokas36/RWK/tree/main/data/PTC

[3] https://huggingface.co/datasets/graphs-datasets/PROTEINS

[4] https://ieeexplore.ieee.org/document/4053093

The sub2vec, which is an unsupervised algorithm for learning the feature representation of arbitrary subgraphs, specifically offers an approach to create a meaningful feature of subgraphs and reasoning. The node2vec is a framework for representational learning for graphs. It can take any graph data learn the continious representation of nodes that can be later on used for different learning tasks.

Semantic precision evaluation

This study primarily asserts that semantic precision can be enhanced through a deep learning model, thus positioning it as a key evaluation metric relative to other works. We utilize the various categories of the above datasets as different semantic classes. That is, we use the compound in each class to quantify the semantic precision of our proposed scheme. Further, we assume that the compounds in different classes are semantically irrelevant for valid semantic precision. Further, the graph search query is to be selected from the same class. We use the semantic precision calculation method used in Gabryel et al. (2018) to evaluate the search precision of our scheme as follows:

$$P = \frac{TP}{TP + FP} \times 100\% \tag{11}$$

Where TP and FP indicate the number of true positive and a false positive return of graphs, respectively.

Sub2vec

Figures 3, 4, 5 and 6 shows its poor performance across all datasets. This is due to its ill-suited subgraph sampling and, subsequently, its vectorization of the given graph, especially for larger graphs. Specifically, sub2vec sample graph substructure with one fixed-length random walk and learn their embedding using several iterations of fixed-length context skip grams for a given graph. The sub2vec random walk sampling strategy is not enough to capture the entire neighbourhoods in the graph, which prevents it from learning structure-equivalent and context-aware representations of an entire graph. This lack of structural similarity is reflected in its subpar performance. This fact is reinforced by the increase in the size of the graph in datasets (i.e., the precision variation in NCI1 and MUTAG datasets). Subfig. (a) 3-6 indicates a linear relationship between m and search precision. This is because a larger number of dimensions in vector embedding preserves latent structural similarity. On the other hand, the low-dimensional vector can not capture structural similarity in the structured datasets. The precision in the subfigure. (b) 3-6 gradually decreases as k increases, as there is a limited number of classes in each dataset; hence, irrelevant results are added to the list.

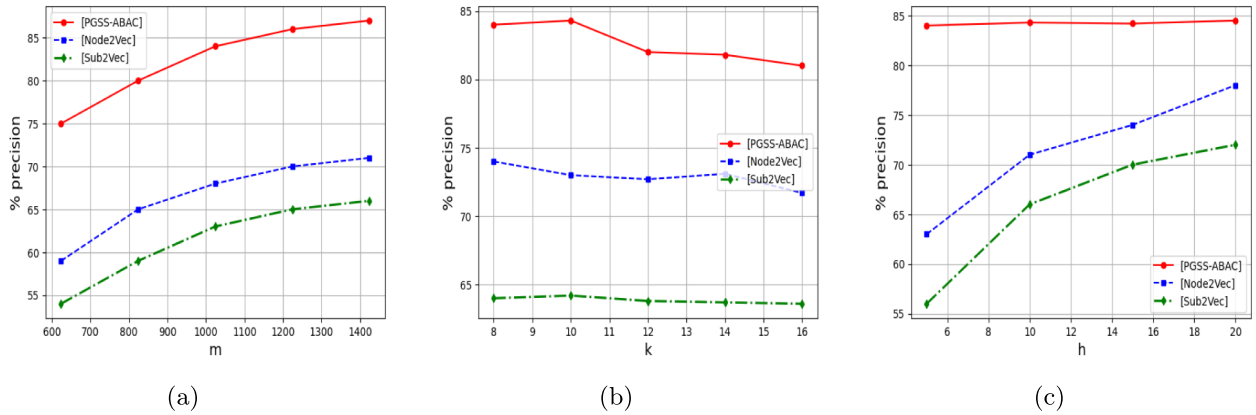


Fig. 3 Semantic precision on the MUTAG dataset, **a** precision w.r.t *m*, **b** precision w.r.t *k*, **c** precision w.r.t *h*

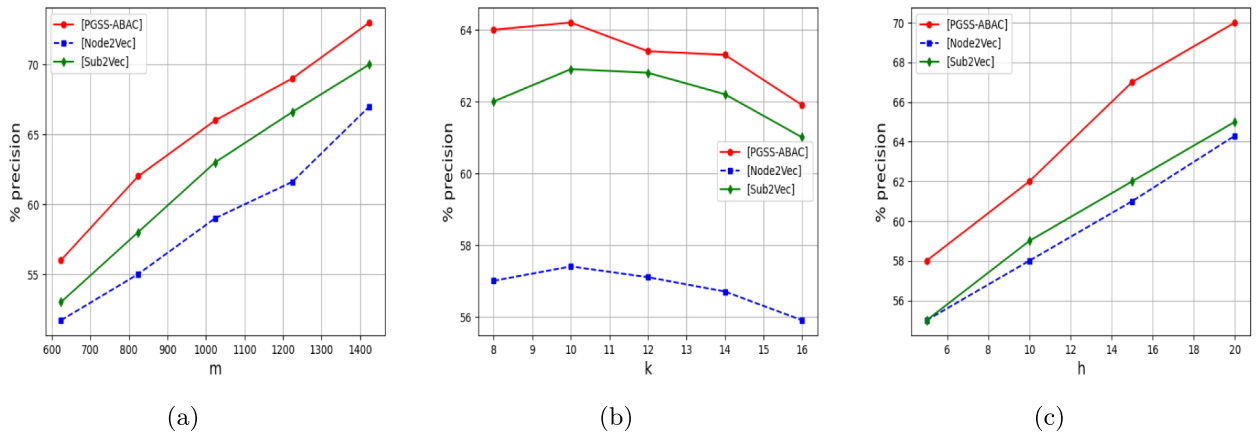


Fig. 4 Semantic precision on the PTC dataset, **a** precision w.r.t *m*, **b** precision w.r.t *k*, **c** precision w.r.t *h*

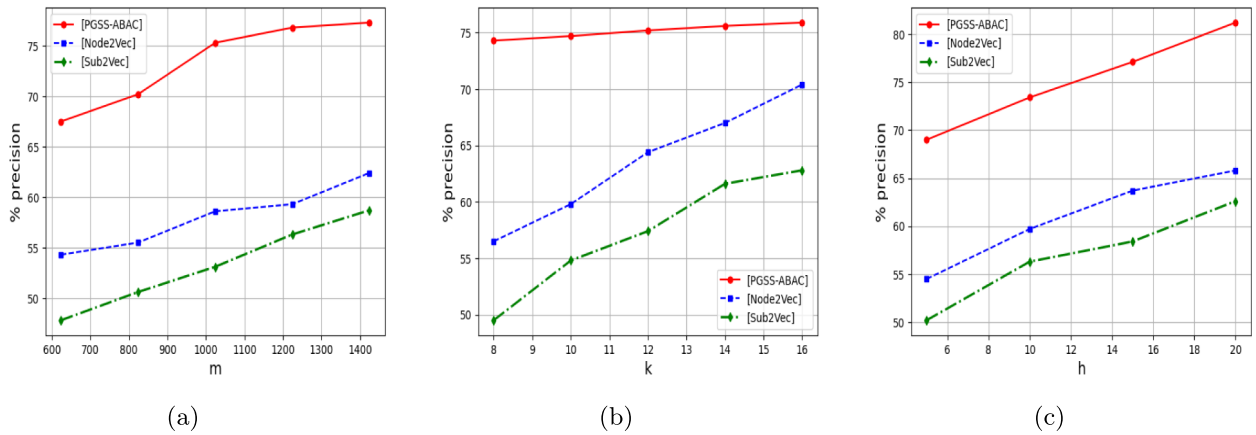


Fig. 5 Semantic precision on the PROTEINS dataset, **a** precision w.r.t *m*, **b** precision w.r.t *k*, **c** precision w.r.t *h*

Moreover, the increase in the value of *h* also helps the sub2vec efficiently capture the latent structural similarity in the query vector, positively influencing the search accuracy.

Node2vec

Results show that the substructure embedding learned by node2vec fails to capture global structure similarities because of its lower order and confined

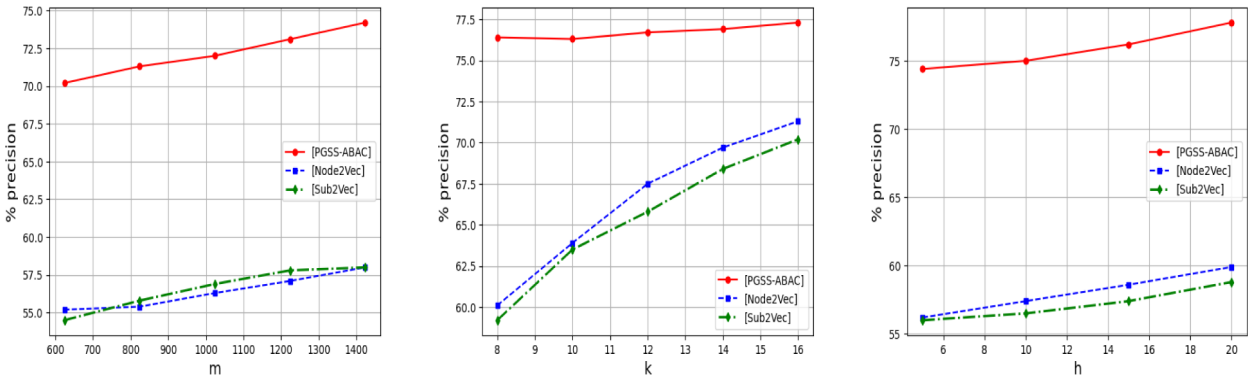


Fig. 6 Semantic precision on the NCI1 dataset, **a** precision w.r.t *m*, **b** precision w.r.t *k*, **c** precision w.r.t *h*

neighbourhood-based model representation technique. These results illustrate that node2vec is not designed to consider global structure identities. This fact is further reinforced by the result of larger datasets, NCI1, and PROTEINS, where its precision is just above 50%. One could conclude that it excels better in substructure analytics tasks (link prediction, node classification, and graph distances Grover and Leskovec 2016). Subfig. (a) 3-6 indicates the influence of m in the feature vector on search accuracy. These figures across all datasets show similar trends. Namely, the search precision steadily increases till the value m reaches 1024, after which the search precision is not notably pronounced. The reason is that the high dimensional feature vector can capture subtle latent structural similarity. On the other hand, low dimension fails to preserve such structural similarity. Subfig. (b) 3-6 indicates the precision loss as k increases. The reason is that non-relevant structured data are ranked as search results, while there is limited relevant structured data available in each class of datasets. Finally, the node2vec precision is significantly positively influenced by the number of nodes in the query feature vector. The reason is that it helps the node2vec to accurately and efficiently infer the latent structured similarity in the query feature vectors.

Proposed PGSS-ABAC scheme

In our proposed work, we utilized the pre-trained Graph2Vec embedding model that captures both global and local similarities among graphs, hence outperforming the traditional kernel-based and representation learning techniques across all datasets. The generic characteristics of the whole graph are captured in the form of their embeddings. Unlike node2Vec (Grover and Leskovec 2016) and Sub2Vec (Adhikari et al. 2017) approaches, which learn graph embedding from linear substructure, our scheme considers and sample rooted subgraphs, a non-linear substructure for learning embedding. Such a non-linear substructure produces similar embedding for similar graphs and preserves structural equivalence. Subfig. (a) 3-6 indicate the growth in the dimension of the feature vector and its influence on search precision. These figures, across all datasets, show similar trends. Namely, the search precision of the PGSS-ABAC scheme is significantly better for higher dimensions than for lower dimensions of the feature vector. The reason is that the high-dimension feature vector can capture a very subtle semantic relationship between subgraphs. On the other hand, when the feature vector does not get enough features, it cannot preserve the structural equivalence of the substructure in the data graph set. Also, the stable part of the figure demonstrates that the subgraph representation beyond a certain m is not notably pronounced in terms of its search precision. Moreover, subfigure. (b) 3-6 shows the precision loss as k increases. This decline is because of a limited number of relevant structured data in each semantic class in the dataset. Hence, more non-relevant data are ranked as search results. Finally, the precision of the PGSS-ABAC scheme is positively influenced by the number of nodes in the query feature vector. The reason is that it helps to efficiently and accurately infer the structure similarity in the query feature vectors.

Search algorithm time cost evaluation

Searching is one of the most frequently utilized operations in SE. Consequently, it is commonly employed as a key parameter for evaluation. The time cost is the time it takes to retrieve the search result from the encrypted database. The cost is of carrying out the secure inner product operation. This section presents the time cost of search algorithms in Cao et al. (2011); Zhu et al. (2019);

Liang and Zhao (2017) based schemes for various settings of n, k, and m.

Search request time w.r.t n

The subfigure. (a) and (b) in Fig. 7 show that the search time of PGSS-ABAC is better than that of Cao et al. (2011), Zhu et al. (2019), and Liang and Zhao (2017) schemes for different values of n and k. The reason is that the PGSS-ABAC index is fixed and smaller than the scale of the graph-structured dataset G. Additionally, introducing a tree-based index structure ignores the secure inner product operation for nodes with fewer similarity scores. The scheme in Cao et al. (2011) uses gSpan (Yan and Han 2002) mining algorithm for frequent subgraph selection. We set the minimum support at the lowest for the features subgraph to get the best performance. However, as demonstrated in Fig. 7a and b, the search request time is almost linear to the number of graph-structured in dataset G. This is because it chooses discriminative subgraphs instead of shrinking processes on a frequent subgraph set. The discriminative subgraphs are (a subset of nodes and edges from the original graph) that can be easily differentiated between graph classes based on their structure. The scheme in Zhu et al. (2019) designed three types of graph indexes with tradeoffs between pruning power, construction cost, and size. The disjoint partition index (DPindex) has the smallest size and the construction time among these indexes. Nevertheless, even for the compact DPindex, as demonstrated in Fig. 7a and b, the search request time is linear to the number of data graphs. The scheme in Liang and Zhao (2017) partitions each graph $g \in G$ into $\tau + k$ subgraphs for index features. Considering the GED threshold τ and the parameter k is a small value. The search time is the worst among the schemes.

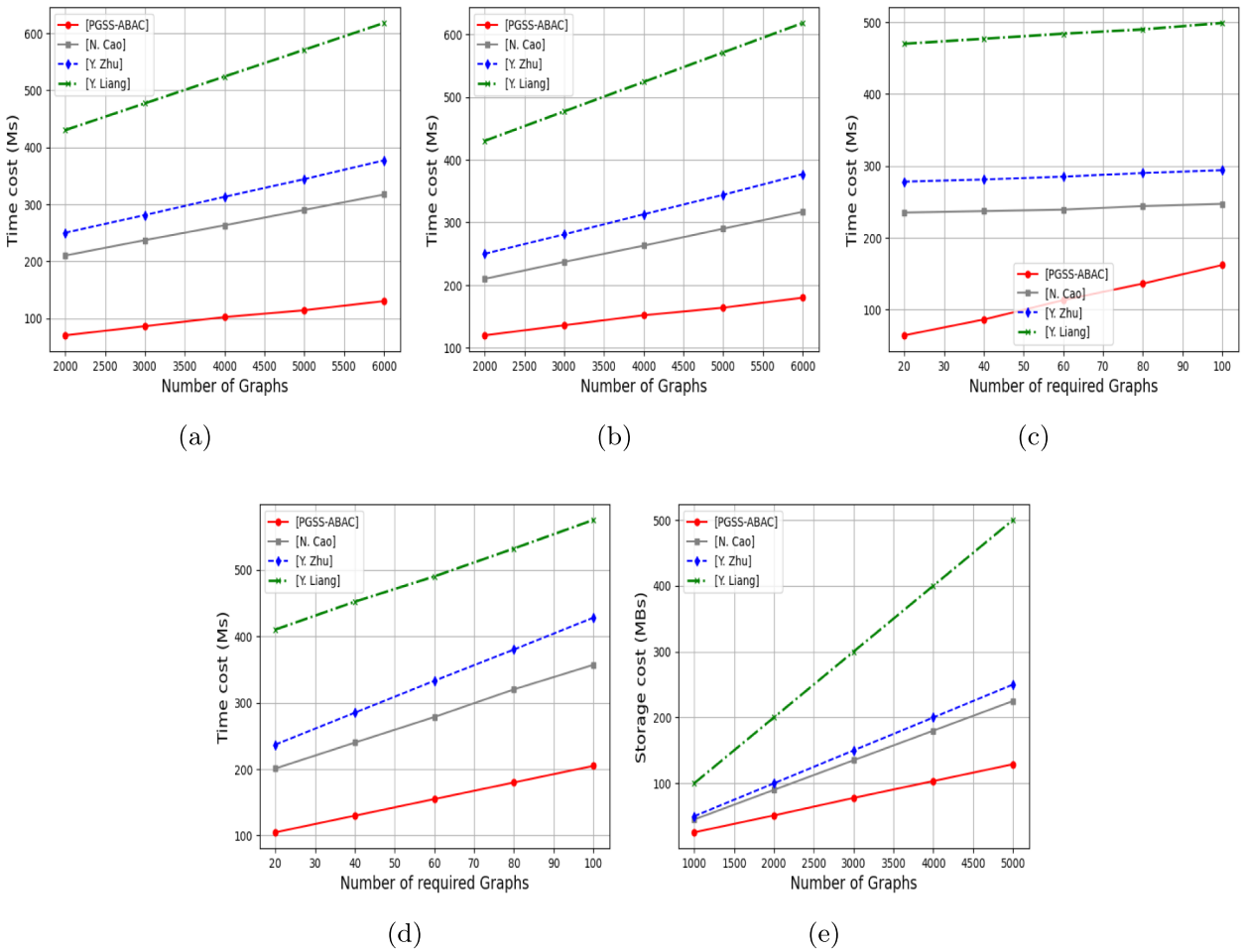


Fig. 7 Time cost w.r.t n, **a** k=40, **b** k=80; Time cost w.r.t k, **c** n=3000, **d** n=6000; Time cost w.r.t m, **e**; Storage cost w.r.t n

Search time w.r.t k

The subfigure. (c) and (d) in Fig. 7 indicates that the search request time of our proposed PGSS-ABAC scheme is better than the rest of the schemes. The reason for this efficiency is the same as mentioned in Section 6.1.1. However, the figures also show that the search time cost of the PGSS-ABAC scheme linearly increases as k increases while the rest of the schemes change little. The reason is that when the GU requests more graph-structured data with the help of parameter k, the chances of accessing the leaf nodes in the index tree also increase. This traversing ensures the retrieval of top-k graph-structure data. However, this extra traversing results in more secure inner product operations, increasing the search request time. However, in the rest of the schemes, the search process needs to traverse all the encrypted graph indexes sequentially, irrespective of the number of requested graph-structured data. Technically, it does not correlate with the number of requested graph-structured data

Search time w.r.t m

Figure 8 shows the correlation between the number of subgraphs in the feature vector and search time for the PGSS-ABAC scheme only. As stated earlier, the index vector in scheme (Cao et al. 2011), Zhu et al. (2019), and Liang and Zhao (2017) are already the whole scale of the graph-structured datasets. The search time of the PGSS-ABAC increases slightly as m increases. The increase in m leads to a more secure inner product operation between the outsourced graph index I and graph query vectors $\tilde{Q}_v$. Hence, the search time positively correlates with the no of subgraphs as features in the index.

Storage cost evaluation

The GU uploads the encrypted structure data and associated secure graph index to realize the structure-aware ranked search. However, we only perform storage cost evaluation from a secure graph index's perspective. Figure 7e indicates that the storage usage of the proposed PGSS-ABAC scheme is better than the rest of the schemes. The reason is that our scheme uses a fixed and compact feature vector, while the rest of the scheme Cao et al. (2011), Zhu et al. (2019), and Liang and Zhao (2017) encompasses the whole scale of the structured-data vocabulary. Therefore, the storage complexity of PGSS-ABAC is $o(nm)$, where the scale of graph dataset vocabulary is larger than m. Hence, the storage cost of the secure graph index in PGSS-ABAC is smaller than that in the rest of the schemes.

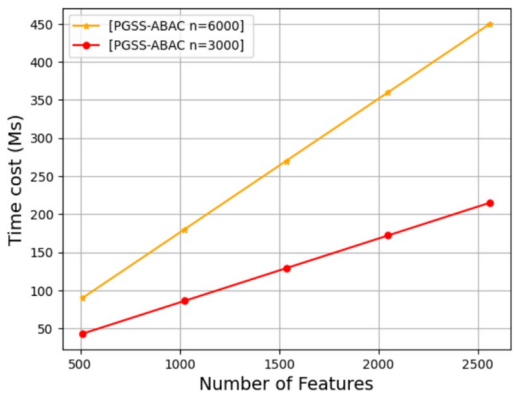


Fig. 8 Time cost w.r.t m, n=3000 and n=6000

Conclusion and future work

In this research, for the first time, we investigate and solve the encrypted full graph similarity search in the M/M setting for cloud computing. We find that the SE on graphs can be structure-aware and return the ranked search results by leveraging the graph neural embedding model. The reuse of shallow layers from the pre-trained Graph2Vec model for the target task, coupled with a novel secure transfer learning, enables the realization of the PGSS-ABAC scheme. The extensive comparative evaluation shows that our scheme outperforms the traditional node and subgraph query approaches in terms of precision and efficiency. Constructing an index tree further improves the search efficiency of our scheme. Moreover, the non-monotonic access policy significantly improved the flexibility of our scheme. The security analysis in two different threat models proves the PGSS-ABAC security. In future work, we will investigate secure search on multi-modality data. Same as text or graph data, the multi-modal data (images, text, audio, and videos) can also be projected into a shared latent space using embedding models; for instance, the OpenAI Contrastive Learning pre-trained (CLIP) model can be used. Different information fusion techniques will be explored in the context of multi-modal searchable encryption. We also plan to investigate the effects of different encryption techniques and how they influence the search efficiency, semantic accuracy, and security of the retrieval process.

Acknowledgements

We would like to express our gratitude to reviewers for their constructive feedback and insightful comments, which significantly contributed to the improvement of this manuscript.

Author Contributions

Shawal Khan: Conceptualization of this study, Methodology, Software, Visualization. Shahzad Khan: Conceptualization, Methodology, Software, Visualization. Weisheng Si: Investigation, Validation, methodology, Supervision, Writing – review & editing. Bahman Javadi: Investigation, methodology, Validation, Supervision.

Funding
This research work received no external funding.

Not applicable.

Declarations

Ethics approval
Not applicable.

Consent for publication
Not applicable.

Competing interests
The authors declare no conflict of interest.

Received: 19 January 2025 Accepted: 21 July 2025
Published online: 11 February 2026

References

Adhikari B, Zhang Y, Ramakrishnan N, Prakash BA (2017) Distributed representations of subgraphs. In: 2017 IEEE International conference on data mining workshops (ICDMW). pp. 111–117. IEEE
Anton H, Rorres C (2013) Elementary linear algebra: applications version. John Wiley & Sons
Bethencourt J, Sahai A, Waters B (2007) Ciphertext-policy attribute-based encryption. In: 2007 IEEE symposium on security and privacy (SP'07). pp. 321–334. IEEE
Boneh D, Boyen X (2004) Efficient selective-id secure identity-based encryption without random oracles. In: International conference on the theory and applications of cryptographic techniques. pp. 223–238. Springer
BULUT M (2016) Reducedcbt and supercbt: two new and improved complete binary tree structures. Turkish J Electric Eng Comput Sci 24(4):2150–2162
Çakmak M (2023) Shallow transfer learning models. In: Engineering sciences in a globalizing world, pp. 53–55. Duvar Publishing, Istanbul, 1st edn
Cao N, Wang C, Li M, Ren K, Lou W (2013) Privacy-preserving multi-keyword ranked search over encrypted cloud data. IEEE Trans Parallel Distrib Syst 25(1):222–233
Cao N, Yang Z, Wang C, Ren K, Lou W (2011) Privacy-preserving query over encrypted graph-structured data in cloud computing. In: 2011 31st International conference on distributed computing systems. pp. 393–402. IEEE
Chase M, Kamara S (2010) Structured encryption and controlled disclosure. In: International conference on the theory and application of cryptology and information security. pp. 577–594. Springer
Chor B (1998) O. goldreich, e. kushilevitz, and m. sudan. private information retrieval. In: 36th annual symposium on foundations of computer science. pp. 41–50
Cordella LP, Foggia P, Sansone C, Vento M (2004) A (sub) graph isomorphism algorithm for matching large graphs. IEEE Trans Pattern Anal Mach Intell 26(10):1367–1372
Dai H, Dai X, Yi X, Yang G, Huang H (2019) Semantic-aware multi-keyword ranked search scheme over encrypted cloud data. J Netw Comput Appl 147:102442
Delfs H, Knebl H (2015) Public-key cryptography. In: Introduction to cryptography, pp. 49–106. Springer
Du M, Wu S, Wang Q, Chen D, Jiang P, Mohaisen A (2020) Graphshield: dynamic large graphs for secure queries with forward privacy. In: IEEE Transactions on Knowledge and Data Engineering
Fan Z, Peng Y, Choi B, Xu J, Bhowmick SS (2013) Towards efficient authenticated subgraph query service in outsourced graph databases. IEEE Trans Serv Comput 7(4):696–713
Fan Z, Choi B, Xu J, Bhowmick SS (2015) Asymmetric structure-preserving subgraph queries for large graphs. In: 2015 IEEE 31st International conference on data engineering. pp. 339–350. IEEE
Gabryel M, Damaševičius R, Przybyszewski K (2018) Application of the bag-of-words algorithm in classification the quality of sales leads. In: International conference on artificial intelligence and soft computing. pp. 615–622. Springer
Gao J, Yu JX, Jin R, Zhou J, Wang T, Yang D (2011) Neighborhood-privacy protected shortest distance computing in cloud. In: Proceedings of the 2011 ACM SIGMOD international conference on management of data. pp. 409–420
Ge X, Yu J, Zhang H, Bai J, Fan J, Xiong NN (2021) Spps: a search pattern privacy system for approximate shortest distance query of encrypted graphs in iiot. IEEE Trans Syst Man Cybern Syst 52(1):136–150
Grover A, Leskovec J (2016) node2vec: Scalable feature learning for networks. In: Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining. pp. 855–864
Han WS, Lee J, Lee JH (2013) Turboiso: towards ultrafast and robust subgraph isomorphism search in large graph databases. In: Proceedings of the 2013 ACM SIGMOD international conference on management of data. pp. 337–348
He X, Vaidya J, Shafiq B, Adam N, Lin X (2010) Reachability analysis in privacy-preserving perturbed graphs. In: 2010 IEEE/WIC/ACM international conference on web intelligence and intelligent agent technology. vol. 1, pp. 691–694. IEEE
Huang L, Xue J, Wang Y, Liu Z, Chen J, Kong Z (2023) Whgdroid: Effective android malware detection based on weighted heterogeneous graph. J Inform Secur Appl 77:103556
Karwa V, Raskhodnikova S, Smith A, Yaroslavtsev G (2011) Private analysis of graph structure. Proceed VLDB Endowment 4(11):1146–1157
Khan S, Abbas H, Iqbal W (2024) Verifiable privacy-preserving image retrieval in multi-owner multi-user settings. IEEE Transactions on emerging topics in computational intelligence
Kundu A, Atallah M, Bertino E (2012) Efficient leakage-free authentication of trees, graphs and forests. Cryptology ePrint Archive
Le Q, Mikolov T (2014) Distributed representations of sentences and documents. In: International conference on machine learning. pp. 1188–1196. PMLR
Liang Y, Zhao P (2017) Similarity search in graph databases: A multi-layered indexing approach. In: 2017 IEEE 33rd international conference on data engineering (ICDE). pp. 783–794. IEEE
Liu Y, Peng W, Su J (2014) A study of ip prefix hijacking in cloud computing networks. Secur Commun Netw 7(11):2201–2210
Mouratidis K, Yiu ML (2012) Shortest path computation with no information leakage. arXiv preprint arXiv:1204.6076
Narayanan A, Chandramohan M, Chen L, Liu Y, Saminathan S (2016) subgraph2vec: Learning distributed representations of rooted sub-graphs from large graphs. arXiv preprint arXiv:1606.08928
Narayanan A, Chandramohan M, Venkatesan R, Chen L, Liu Y, Jaiswal S (2017) graph2vec: Learning distributed representations of graphs. arXiv preprint arXiv:1707.05005
Ostrovsky R, Sahai A, Waters B (2007) Attribute-based encryption with non-monotonic access structures. In: Proceedings of the 14th ACM conference on computer and communications security. pp. 195–203
Řehůřek R, Sojka P (May 2010) Software Framework for Topic Modelling with Large Corpora. In: Proceedings of the LREC 2010 workshop on new challenges for NLP frameworks. pp. 45–50. ELRA, Valletta, Malta
Sahai A, Waters B (2005) Fuzzy identity-based encryption. In: Annual international conference on the theory and applications of cryptographic techniques. pp. 457–473. Springer
Shang H, Zhang Y, Lin X, Yu JX (2008) Taming verification hardness: an efficient algorithm for testing subgraph isomorphism. Proceed VLDB Endowment 1(1):364–375
Song DX, Wagner D, Perrig A (2000) Practical techniques for searches on encrypted data. In: Proceeding 2000 IEEE symposium on security and privacy. S &P 2000. pp. 44–55. IEEE
Sun L, Xu C, Zhang Y (2018) A dynamic and non-interactive Boolean searchable symmetric encryption in multi-client setting. J Inform Secur Appl 40:145–155
Sun W, Wang B, Cao N, Li M, Lou W, Hou YT, Li H (2013) Privacy-preserving multi-keyword text search in the cloud supporting similarity-based ranking. In: Proceedings of the 8th ACM SIGSAC symposium on information, computer and communications security. pp. 71–82
Ullmann JR (1976) An algorithm for subgraph isomorphism. J ACM (JACM) 23(1):31–42

Wang S, Zheng Y, Jia X, Yi X (2022) Pegraph: A system for privacy-preserving and efficient search over encrypted social graphs. IEEE Trans Inf Forensics Secur 17:3179–3194
Wang S, Zheng Y, Jia X, Huang H, Wang C (2023) Prigsim: towards privacy-preserving graph similarity search as a cloud service. IEEE Trans Knowl Data Eng 35(10):10478–10496
Wong WK, Cheung DWl, Kao B, Mamoulis N (2009) Secure knn computation on encrypted databases. In: Proceedings of the 2009 ACM SIGMOD international conference on management of data. pp. 139–152
Yanardag P, Vishwanathan S (2015) Deep graph kernels. In: Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining. pp. 1365–1374
Yan X, Han J (2002) gspan: graph-based substructure pattern mining. In: 2002 IEEE international conference on data mining, 2002. Proceedings. pp. 721–724 . https://doi.org/10.1109/ICDM.2002.1184038
Yan X, Yu PS, Han J (2004) Graph indexing: a frequent structure-based approach. In: Proceedings of the 2004 ACM SIGMOD international conference on management of data. pp. 335–346
Yi P, Fan Z, Yin S (2014) Privacy-preserving reachability query services for sparse graphs. In: 2014 IEEE 30th international conference on data engineering workshops. pp. 32–35. IEEE
Zhang C, Zhu L, Xu C, Sharif K, Zhang C, Liu X (2020) Pgas: Privacy-preserving graph encryption for accurate constrained shortest distance queries. Inf Sci 506:325–345
Zhu Y, Qin L, Yu JX, Cheng H (2019) Answering top-*k* graph similarity queries in graph databases. IEEE Trans Knowl Data Eng 32(8):1459–1474
Zhu H, Wu B, Xie M, Cui Z, Wu Z (2016) Secure shortest path search over encrypted graph supporting synonym query in cloud computing. In: 2016 IEEE Trustcom/BigDataSE/ISPA. pp. 497–504. IEEE
Zuo X, Li L, Peng H, Luo S, Yang Y (2020) Privacy-preserving subgraph matching scheme with authentication in social networks. IEEE Transactions on cloud computing

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.